Guion de presentación y demostración

Material sin portada. Duración sugerida: 8–10 minutos. La portada de la estudiante se añade por separado.

Secuencia de exposición

Tiempo	Tema	Qué explicar o mostrar
0:00–0:45	Producto y objetivo	Juego web 2D de tablero. La entrega integra interacción entre dos jugadores en una sala.
0:45–2:00	Arquitectura	Netlify entrega la interfaz; Render mantiene el estado. Cada navegador tiene su propia conexión con Socket.IO.
2:00–3:00	Autoridad y sincronización	Los clientes solicitan acciones. El servidor valida turnos, genera el dado y difunde `room-updated`.
3:00–6:30	Demostración con dos clientes	Crear, compartir código, unirse, elegir personajes, iniciar, lanzar y comparar ambos estados. Resolver un reto compartido si aparece.
6:30–7:30	Pruebas	Explicar el resumen de resultados, dos clientes reales de navegador y relojes simulados para vencimientos.
7:30–8:30	Limitaciones y futuro	Las salas viven en memoria; todavía no se recuperan partidas al desconectar. Próximos pasos: persistencia, recuperación y medición de carga.
8:30–10:00	Preguntas	Usar las respuestas técnicas preparadas al final.

Demostración principal: dos personas conectadas

1. Abrir https://videojuego3d.netlify.app/ en dos dispositivos o dos páginas de navegador. No usar una publicación antigua fijada a otro despliegue.
2. En el primero, pulsar **Crear partida**, escribir un nombre y crear la sala.
3. En el segundo, pulsar **Unirse a partida**, escribir otro nombre e introducir el mismo código.
4. Elegir personajes diferentes. Mostrar que la selección y los jugadores aparecen en ambas pantallas.
5. Desde el creador, pulsar **Comenzar aventura**.
6. Lanzar el dado en el cliente que tiene el turno. Comparar posición, dado, monedas y turno en ambas pantallas. El rival no puede lanzar fuera de turno.

7. Si aparece **Duelo contra rival**, contestar primero desde un cliente y mostrar que su opción queda bloqueada mientras el otro todavía puede responder. Después contestar desde el segundo y comparar los resultados.
8. Si el tiempo lo permite, reunir monedas, comprar un escudo o una pista y mostrar su activación en **Tus hechizos**, encima de las respuestas. El escudo solo sirve en duelos y se activa antes de contestar.
9. Abrir el código en `server/index.cjs` y señalar la validación y el envío de `room-updated`; en `server/challenges.cjs`, señalar el plazo y la vista pública del reto.

El dado de producción es aleatorio. No se puede garantizar que aparezca un duelo en los primeros lanzamientos. Si no aparece dentro del tiempo de exposición, continuar la demostración de sincronización y explicar el caso con las capturas o las pruebas automatizadas. El dado determinista se usa únicamente en el entorno de pruebas, y debe identificarse como tal.

Preparación antes de la clase

- Comprobar el acceso desde la red del aula y abrir el servidor unos minutos antes, sin instalar un servicio que haga peticiones permanentes.
- Tener preparados dos clientes humanos. El modo contra el bot no sustituye la demostración de interacción entre dos personas.
- Confirmar que el código entregado sea la rama `codex/mobile-visual-improvements` o el ZIP correspondiente.
- Llevar el informe, la presentación editable, su PDF, el código fuente y las evidencias de pruebas.
- Para una demostración local, preparar `npm ci` y las dependencias de Playwright con antelación. El juego normal no necesita Playwright para ejecutarse.
- No recargar ni cerrar una sesión durante una partida que se quiera conservar. Actualmente no existe recuperación de la partida al reconectar.

Alternativa local si el acceso a Internet falla

Desde la carpeta del proyecto, arrancar `npm run server` en una terminal y `npm run dev -- --host 0.0.0.0` en otra. Usar dos páginas del navegador en la misma computadora. Si se utilizarán dos dispositivos en una red local, configurar con antelación `VITE_SERVER_URL` con la dirección del servidor y permitir el origen del frontend en `server/origins.cjs`.

Una demostración local con dos clientes sigue mostrando intercambio por Socket.IO, pero no demuestra por sí sola el acceso mediante la red pública de la escuela. Indicar esta diferencia al presentarla.

Respuestas para la revisión del código

¿Por qué Socket.IO? Ofrece eventos, salas, acuses y reconexión. Puede usar WebSocket y sondeo HTTP prolongado, según la conexión disponible.

¿Quién decide el resultado del dado y las monedas? El servidor. El navegador solicita la acción y dibuja el estado público recibido.

¿Por qué el tiempo no depende del reloj del celular? El servidor registra y valida el plazo de respuesta. La cuenta atrás del navegador es una representación visual.

¿Puede el rival ver mi respuesta antes de terminar? El estado público indica que respondí, pero no revela la opción ni la solución hasta los resultados. Las pistas se devuelven en un acuse privado.

¿Qué pasa si alguien se desconecta? Se cancela el reto. En una sala compartida se pierde la partida activa y se vuelve a esperar participantes; una sala individual se elimina. Reconectar el socket no recupera esa partida.

¿Se probaron muchos jugadores a la vez? No se hizo una prueba de carga. Los resultados demuestran los casos funcionales y resoluciones incluidos, no un límite de concurrencia del servidor.

¿Por qué React y no un motor como Unity o Godot? Es un juego de tablero 2D basado en interfaz web y SVG. React gestiona sus pantallas y Node.js con Socket.IO resuelve la comunicación en red. La consigna recomienda herramientas, pero no exige un motor concreto en el texto facilitado.

¿El bot utiliza una IA externa? No. Es un rival programado con probabilidades y pausas según la dificultad. Utiliza las mismas acciones y reglas del servidor.

¿Las pruebas son partidas reales completas? Hay pruebas de lógica con reloj simulado, escenarios visuales y recorridos reales de interfaz con servidores locales. La victoria completa del bot se comprueba con reloj simulado; no se ha medido una partida completa de dos humanos en cada dispositivo físico.